

lab2

February 2, 2025

1 Data Modeling for a Blogging Platform

1.1 Objectives

This report outlines the design and implementation of a database system for a blogging platform, focusing on data modelling techniques and query execution. The objective is to develop a structured database that efficiently handles user accounts, blog posts, and comments while ensuring data integrity and scalability. The report details the conceptual design, schema translation, implementation using MongoDB, and an evaluation of SQL vs. NoSQL database choices.

The primary objectives of this assignment are as follows:

1. **Data Modeling**
 - Analyze the dataset and identify key entities and relationships.
 - Develop an **Entity-Relationship Diagram (ERD)** to represent the data structure conceptually.
2. **Database Schema Implementation**
 - Translate the conceptual model into a **physical database schema**.
 - Implement the schema in **MongoDB**, leveraging its document-based structure.
3. **Executing Queries and Data Operations**
 - Retrieve data efficiently, including all users, posts, and comments.
 - Perform CRUD (Create, Read, Update, Delete) operations such as updating user emails and deleting comments.
 - Insert a new user and analyze any potential conflicts or integrity issues.
4. **SQL vs. NoSQL Analysis**
 - Evaluate the choice of **MongoDB over PostgreSQL** for this dataset.
 - Identify the benefits and limitations of using NoSQL versus SQL in this use case.
5. **Comprehensive Report Documentation**
 - Provide an **executive summary** of the database design and implementation process.
 - Include **ERD diagrams, schema screenshots, query results, and insights**.

1.2 Approach and Methodology

The assignment follows a structured methodology:

1. **Dataset Analysis:**
 - Review the provided dataset and identify key attributes.
 - Determine relationships between users, posts, and comments.
2. **Conceptual Modeling:**
 - Develop an **Entity-Relationship Diagram (ERD)** to visually represent relationships.

3. Schema Design & Database Selection:

- Justify the selection of **MongoDB** as the preferred database.
- Design the **physical schema** using collections and embedded documents where applicable.

4. Query Implementation:

- Implement and execute queries in MongoDB.
- Perform key data operations such as insert, update, delete, and retrieval.

5. Evaluation & Discussion:

- Compare MongoDB's document-based approach to PostgreSQL's relational model.
- Discuss advantages and potential trade-offs.

6. Report Compilation:

- Document the database design, implementation process, and key observations.
- Provide screenshots and output samples of query execution.

1.3 Expected Outcomes

Upon completing this assignment, the following outcomes are anticipated:

- A **well-structured database model** optimized for managing users, blog posts, and comments.
- A **functional implementation in MongoDB**, demonstrating key CRUD operations.
- A **detailed comparison of SQL vs. NoSQL**, providing insights into database selection criteria.
- A **comprehensive report** including the conceptual model, database schema, queries, and execution results.

1.4 Step 1: Dataset Analysis & Entity-Relationship Diagram (ERD)

1.4.1 Dataset Analysis

The dataset consists of three main entities:

1. Users

- Each user can create multiple blog posts.
- Attributes: `user_id`, `username`, `email`, `password`, `created_at`.

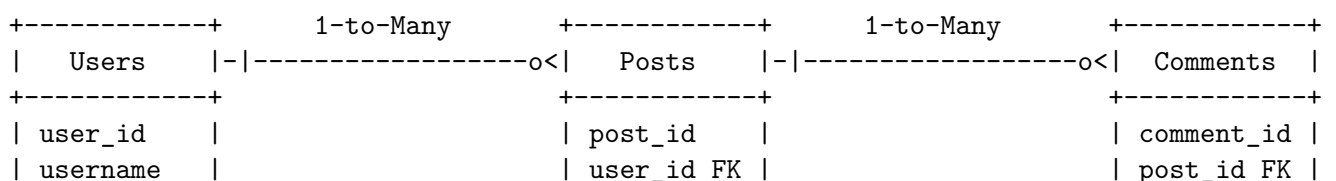
2. Posts

- Each post is authored by a user.
- Each post can have multiple comments.
- Attributes: `post_id`, `user_id` (FK), `title`, `content`, `updated_at`.

3. Comments

- Each comment is associated with a specific post.
- Attributes: `comment_id`, `post_id` (FK), `content`, `created_at`.

1.4.2 Entity-Relationship Diagram (ERD)



email		title		content	
password		content		created_at	
created_at		updated_at		+	+
+-----+		+-----+			

An **Entity-Relationship Diagram (ERD)** has been created to represent the relationships between these entities visually. The ERD illustrates the following relationships:

- **Users → Posts (1-to-Many)**: One user can write multiple posts. A user may not have a post but a post must have a user hence cardinality is mandatory one to optional many
- **Posts → Comments (1-to-Many)**: Each post can have multiple comments. A post may not have a comment but a comment must be in a post hence cardinality is mandatory one to optional many
- **Users do not own comments** in this dataset; comments are anonymous otherwise multiple users can comment on multiple posts hence **Many-to-Many** relationship.

1.5 Step 2: Translating the Conceptual Model to a Physical Model

1.5.1 Choice of Database Management System (DBMS)

For this project, **MongoDB (NoSQL)** has been chosen as the database system. The decision is based on the following factors:

1. **Schema Flexibility:**
 - The dataset contains nested attributes, particularly in the `comments` field.
 - MongoDB allows embedding `comments` within `posts`, reducing the need for joins.
2. **Performance & Scalability:**
 - MongoDB provides faster read and write operations for high-traffic applications.
 - NoSQL databases scale horizontally, making them ideal for growing datasets.
3. **Simplified Queries:**
 - Queries for retrieving a post along with its comments are more efficient in MongoDB.
 - The document-based structure eliminates complex JOIN operations.
4. **Ease of Evolution:**
 - MongoDB allows schema changes without requiring major structural modifications.
 - New fields (e.g., `tags`, `likes`) can be added to posts dynamically.

1.5.2 Physical Database Schema in MongoDB

Sample Users Collection (`users`)

```
{
  "user_id": "uid1",
  "username": "sarah_c",
  "email": "sarah@example.com",
  "password": "sarapass",
  "created_at": "2024-01-16 08:10"
}
```

Sample Posts Collection (`posts`)

```

{
  "post_id": "pid3",
  "user_id": "uid1",
  "title": "JavaScript Basics",
  "content": "JavaScript is a popular...",
  "updated_at": "2024-01-16 09:30",
  "comments": [
    {
      "comment_id": "cid5",
      "content": "JavaScript is amazing!",
      "created_at": "2024-01-16 08:30"
    }
  ]
}

```

Sample Comments Collection (posts)

```

{
  "comment_id": "cid5",
  "post_id": "pid3",
  "content": "JavaScript is amazing!",
  "created_at": "2024-01-16 08:30"
}

```

1.5.3 Justification for MongoDB Over PostgreSQL

Feature	MongoDB (NoSQL)	PostgreSQL (SQL)
Schema Design	Flexible, documents evolve dynamically	Fixed schema with strict table definitions
Data Storage	JSON-like documents (BSON)	Relational tables with predefined columns
Querying	Fast document retrieval	Complex joins for related data
Scalability	Horizontal scaling (sharding)	Vertical scaling (index optimization)
Best for	Nested & unstructured data	Structured, transactional data

Given the semi-structured nature of comments and the need for efficient post-retrieval, **MongoDB is the optimal choice** for this use case.

1.5.4 The following script connects to MongoDB and extracts users, posts, and comments from the provided CSV file:

```
[22]: import pandas as pd
```

```
# Load the dataset
file_path = "data/blogging_dataset.csv"
df = pd.read_csv(file_path, encoding='ISO-8859-1')

df
```

```
[22]:  user_id  username          email      password      created_at \
0   uid1    sarah_c    sarah@example.com    sarapass  2024-01-16 08:10
1   uid2    emily_j    emily@example.com    secure123  2024-01-16 17:30
2   uid3    jane_s     jane@example.com    passw0rd  2024-01-14 14:45
3   uid4  michael_b  michael@example.com    p@ssw0rd  2024-01-16 12:20
4   uid5    john_d     john@example.com  password123  2024-01-15 09:30
```

```
      post_id          content          title \
0      pid3  JavaScript is a popular  JavaScript Basics
1      pid5  Build web applications using  Python Web Development
2      pid1  Python is a versatile and  Getting Started with Python
3      pid4  Learn about various data  Data Structures in C++
4      pid2  MongoDB is a highly scalable  Introduction to MongoDB
```

```
      upated_at          comments
0  2024-01-16 09:30  JavaScript is amazing! 2024-01-16 08:30 cid5
1  2024-01-16 18:00  Thanks for the tutorial! 2024-01-16 18:00 cid7
2  2024-01-15 10:20  I learned a lot. 2024-01-14 16:00 cid2 | Thank...
3  2024-01-16 15:45  Great explanation! 2024-01-16 13:00 cid6
4  2024-01-15 09:35  Well explaiend. 2024-01-15 10:00 cid4 | Great ...
```

```
[23]: # Extract users
```

```
users_df = df[['user_id', 'username', 'email', 'password', 'created_at']].
↳drop_duplicates().sort_values(by="user_id", ascending=True)
users_df
```

```
[23]:  user_id  username          email      password      created_at
0   uid1    sarah_c    sarah@example.com    sarapass  2024-01-16 08:10
1   uid2    emily_j    emily@example.com    secure123  2024-01-16 17:30
2   uid3    jane_s     jane@example.com    passw0rd  2024-01-14 14:45
3   uid4  michael_b  michael@example.com    p@ssw0rd  2024-01-16 12:20
4   uid5    john_d     john@example.com  password123  2024-01-15 09:30
```

```
[24]: # Extract posts
posts_df = df[['post_id', 'user_id', 'title', 'content', 'updated_at']].
↳drop_duplicates().sort_values(by="post_id", ascending=True)
posts_df
```

```
[24]:
```

	post_id	user_id	title \	content	updated_at
2	pid1	uid3	Getting Started with Python	Python is a versatile and	2024-01-15 10:20
4	pid2	uid5	Introduction to MongoDB	MongoDB is a highly scalable	2024-01-15 09:35
0	pid3	uid1	JavaScript Basics	JavaScript is a popular	2024-01-16 09:30
3	pid4	uid4	Data Structures in C++	Learn about various data	2024-01-16 15:45
1	pid5	uid2	Python Web Development	Build web applications using	2024-01-16 18:00

```
[25]: # Extract comments
comments_list = []
for _, row in df.iterrows():
    comments = str(row['comments']).split('|')
    for comment in comments:
        parts = comment.strip().split(' ')
        if len(parts) >= 2:
            comment_content = ' '.join(parts[:-2])
            created_at = ' '.join(parts[-2:])
            comment_id = parts[-1]
            comments_list.append({"comment_id": comment_id, "post_id":
↳row['post_id'], "content": comment_content, "created_at": created_at})

sorted_comments = sorted(comments_list, key=lambda comment:
↳comment["comment_id"])
sorted_comments
```

```
[25]: [{'comment_id': 'cid1',
        'post_id': 'pid1',
        'content': 'Thanks for the tutorial! 2024-01-14',
        'created_at': '15:30 cid1'},
        {'comment_id': 'cid2',
        'post_id': 'pid1',
```

```

        'content': 'I learned a lot. 2024-01-14',
        'created_at': '16:00 cid2'},
{'comment_id': 'cid3',
 'post_id': 'pid2',
 'content': 'Great article! 2024-01-15',
 'created_at': '09:45 cid3'},
{'comment_id': 'cid4',
 'post_id': 'pid2',
 'content': 'Well explaind. 2024-01-15',
 'created_at': '10:00 cid4'},
{'comment_id': 'cid5',
 'post_id': 'pid3',
 'content': 'JavaScript is amazing! 2024-01-16',
 'created_at': '08:30 cid5'},
{'comment_id': 'cid6',
 'post_id': 'pid4',
 'content': 'Great explanation! 2024-01-16',
 'created_at': '13:00 cid6'},
{'comment_id': 'cid7',
 'post_id': 'pid5',
 'content': 'Thanks for the tutorial! 2024-01-16',
 'created_at': '18:00 cid7'}]

```

```

[28]: import os
      from pymongo import MongoClient

      # Fetch configuration from the environment, with defaults if not set.
      mongo_uri = os.environ.get("MONGODB_URI", "mongodb://localhost:27017/")
      db_name = os.environ.get("MONGODB_DB", "blogging_platform")

      # Connect to MongoDB
      client = MongoClient(mongo_uri)
      db = client[db_name]
      print("Current database name:", db.name)

      users_collection = db["users"]
      posts_collection = db["posts"]
      comments_collection = db["comments"]

```

Current database name: blogging_platform

```

[30]: # Insert users into MongoDB
      users_records = users_df.to_dict(orient='records')
      users_collection.insert_many(users_records)
      print("Users inserted:", users_collection.count_documents({}))

```

Users inserted: 5

```
[31]: # Insert posts into MongoDB
posts_records = posts_df.rename(columns={'upated_at': 'updated_at'}).
    ↪to_dict(orient='records')
posts_collection.insert_many(posts_records)
print("Posts inserted:", posts_collection.count_documents({}))
```

Posts inserted: 5

```
[32]: # Insert comments into MongoDB
comments_collection.insert_many(sorted_comments)
print("Comments inserted:", comments_collection.count_documents({}))
```

Comments inserted: 7

```
[33]: # Show all collections in the database
print("Collections in the database:", db.list_collection_names())
```

Collections in the database: ['posts', 'users', 'comments']

The above scripts: - Reads the dataset from the CSV file. - Connects to **MongoDB** and creates three separate collections: **users**, **posts**, and **comments**. - Extracts and processes data from the CSV file. - Inserts structured records into **MongoDB**. - Confirms data insertion by counting documents in each collection.

1.6 Step 3: MongoDB Query Implementation

The following queries are executed in **MongoDB** to manipulate and retrieve data.

```
[34]: # a. Retrieve all users
users = users_collection.find()
for user in users:
    print(user)
```

```
{'_id': ObjectId('67a01b6ef2c3875d00315832'), 'user_id': 'uid1', 'username':
'sarah_c', 'email': 'sarah@example.com', 'password': 'sarapass', 'created_at':
'2024-01-16 08:10'}
{'_id': ObjectId('67a01b6ef2c3875d00315833'), 'user_id': 'uid2', 'username':
'emily_j', 'email': 'emily@example.com', 'password': 'secure123', 'created_at':
'2024-01-16 17:30'}
{'_id': ObjectId('67a01b6ef2c3875d00315834'), 'user_id': 'uid3', 'username':
'jane_s', 'email': 'jane@example.com', 'password': 'passw0rd', 'created_at':
'2024-01-14 14:45'}
{'_id': ObjectId('67a01b6ef2c3875d00315835'), 'user_id': 'uid4', 'username':
'michael_b', 'email': 'michael@example.com', 'password': 'p@ssw0rd',
'created_at': '2024-01-16 12:20'}
{'_id': ObjectId('67a01b6ef2c3875d00315836'), 'user_id': 'uid5', 'username':
'john_d', 'email': 'john@example.com', 'password': 'password123', 'created_at':
'2024-01-15 09:30'}
```



```
[35]: # b. Retrieve all posts
posts = posts_collection.find()
for post in posts:
    print(post)

{'_id': ObjectId('67a01b6ff2c3875d00315837'), 'post_id': 'pid1', 'user_id':
'uid3', 'title': 'Getting Started with Python', 'content': 'Python is a versatile
and \x85', 'updated_at': '2024-01-15 10:20'}
{'_id': ObjectId('67a01b6ff2c3875d00315838'), 'post_id': 'pid2', 'user_id':
'uid5', 'title': 'Introduction to MongoDB', 'content': 'MongoDB is a highly
scalable \x85', 'updated_at': '2024-01-15 09:35'}
{'_id': ObjectId('67a01b6ff2c3875d00315839'), 'post_id': 'pid3', 'user_id':
'uid1', 'title': 'JavaScript Basics', 'content': 'JavaScript is a popular \x85',
'updated_at': '2024-01-16 09:30'}
{'_id': ObjectId('67a01b6ff2c3875d0031583a'), 'post_id': 'pid4', 'user_id':
'uid4', 'title': 'Data Structures in C++', 'content': 'Learn about various data
\x85', 'updated_at': '2024-01-16 15:45'}
{'_id': ObjectId('67a01b6ff2c3875d0031583b'), 'post_id': 'pid5', 'user_id':
'uid2', 'title': 'Python Web Development', 'content': 'Build web applications
using \x85', 'updated_at': '2024-01-16 18:00'}
```

```
[36]: # c. Retrieve all comments
comments = comments_collection.find()
for comment in comments:
    print(comment)

{'_id': ObjectId('67a01b6ff2c3875d0031583c'), 'comment_id': 'cid1', 'post_id':
'pid1', 'content': 'Thanks for the tutorial! 2024-01-14', 'created_at': '15:30
cid1'}
{'_id': ObjectId('67a01b6ff2c3875d0031583d'), 'comment_id': 'cid2', 'post_id':
'pid1', 'content': 'I learned a lot. 2024-01-14', 'created_at': '16:00 cid2'}
{'_id': ObjectId('67a01b6ff2c3875d0031583e'), 'comment_id': 'cid3', 'post_id':
'pid2', 'content': 'Great article! 2024-01-15', 'created_at': '09:45 cid3'}
{'_id': ObjectId('67a01b6ff2c3875d0031583f'), 'comment_id': 'cid4', 'post_id':
'pid2', 'content': 'Well explained. 2024-01-15', 'created_at': '10:00 cid4'}
{'_id': ObjectId('67a01b6ff2c3875d00315840'), 'comment_id': 'cid5', 'post_id':
'pid3', 'content': 'JavaScript is amazing! 2024-01-16', 'created_at': '08:30
cid5'}
{'_id': ObjectId('67a01b6ff2c3875d00315841'), 'comment_id': 'cid6', 'post_id':
'pid4', 'content': 'Great explanation! 2024-01-16', 'created_at': '13:00 cid6'}
{'_id': ObjectId('67a01b6ff2c3875d00315842'), 'comment_id': 'cid7', 'post_id':
'pid5', 'content': 'Thanks for the tutorial! 2024-01-16', 'created_at': '18:00
cid7'}
```

```
[37]: # d. Update the user email of `sarah_c` to `sarab@gmail.com`
users_collection.update_one({"username": "sarah_c"}, {"$set": {"email": "
↪sarab@gmail.com"}})
users = users_collection.find()
```

```
for user in users:
    print(user)
```

```
{'_id': ObjectId('67a01b6ef2c3875d00315832'), 'user_id': 'uid1', 'username':
'sarah_c', 'email': 'sarab@gmail.com', 'password': 'sarapass', 'created_at':
'2024-01-16 08:10'}
{'_id': ObjectId('67a01b6ef2c3875d00315833'), 'user_id': 'uid2', 'username':
'emily_j', 'email': 'emily@example.com', 'password': 'secure123', 'created_at':
'2024-01-16 17:30'}
{'_id': ObjectId('67a01b6ef2c3875d00315834'), 'user_id': 'uid3', 'username':
'jane_s', 'email': 'jane@example.com', 'password': 'passw0rd', 'created_at':
'2024-01-14 14:45'}
{'_id': ObjectId('67a01b6ef2c3875d00315835'), 'user_id': 'uid4', 'username':
'michael_b', 'email': 'michael@example.com', 'password': 'p@ssw0rd',
'created_at': '2024-01-16 12:20'}
{'_id': ObjectId('67a01b6ef2c3875d00315836'), 'user_id': 'uid5', 'username':
'john_d', 'email': 'john@example.com', 'password': 'password123', 'created_at':
'2024-01-15 09:30'}
```

[38]: *# e. Delete comment with `comment\_id = cid4`*

```
comments_collection.delete_one({"comment_id": "cid4"})
comments = comments_collection.find()
for comment in comments:
    print(comment)
```

```
{'_id': ObjectId('67a01b6ff2c3875d0031583c'), 'comment_id': 'cid1', 'post_id':
'pid1', 'content': 'Thanks for the tutorial! 2024-01-14', 'created_at': '15:30
cid1'}
{'_id': ObjectId('67a01b6ff2c3875d0031583d'), 'comment_id': 'cid2', 'post_id':
'pid1', 'content': 'I learned a lot. 2024-01-14', 'created_at': '16:00 cid2'}
{'_id': ObjectId('67a01b6ff2c3875d0031583e'), 'comment_id': 'cid3', 'post_id':
'pid2', 'content': 'Great article! 2024-01-15', 'created_at': '09:45 cid3'}
{'_id': ObjectId('67a01b6ff2c3875d00315840'), 'comment_id': 'cid5', 'post_id':
'pid3', 'content': 'JavaScript is amazing! 2024-01-16', 'created_at': '08:30
cid5'}
{'_id': ObjectId('67a01b6ff2c3875d00315841'), 'comment_id': 'cid6', 'post_id':
'pid4', 'content': 'Great explanation! 2024-01-16', 'created_at': '13:00 cid6'}
{'_id': ObjectId('67a01b6ff2c3875d00315842'), 'comment_id': 'cid7', 'post_id':
'pid5', 'content': 'Thanks for the tutorial! 2024-01-16', 'created_at': '18:00
cid7'}
```

[39]: *# f. Insert a new user and analyze potential conflicts***

```
new_user = {
    "user_id": "uid5",
    "username": "John_d",
    "email": "john@go.com",
```

```

        "password": "password123",
        "created_at": "2024-01-16 10:00"
    }
    users_collection.insert_one(new_user)
    users = users_collection.find()
    for user in users:
        print(user)

```

```

{'_id': ObjectId('67a01b6ef2c3875d00315832'), 'user_id': 'uid1', 'username':
'sarah_c', 'email': 'sarab@gmail.com', 'password': 'sarapass', 'created_at':
'2024-01-16 08:10'}
{'_id': ObjectId('67a01b6ef2c3875d00315833'), 'user_id': 'uid2', 'username':
'emily_j', 'email': 'emily@example.com', 'password': 'secure123', 'created_at':
'2024-01-16 17:30'}
{'_id': ObjectId('67a01b6ef2c3875d00315834'), 'user_id': 'uid3', 'username':
'jane_s', 'email': 'jane@example.com', 'password': 'passw0rd', 'created_at':
'2024-01-14 14:45'}
{'_id': ObjectId('67a01b6ef2c3875d00315835'), 'user_id': 'uid4', 'username':
'michael_b', 'email': 'michael@example.com', 'password': 'p@ssw0rd',
'created_at': '2024-01-16 12:20'}
{'_id': ObjectId('67a01b6ef2c3875d00315836'), 'user_id': 'uid5', 'username':
'john_d', 'email': 'john@example.com', 'password': 'password123', 'created_at':
'2024-01-15 09:30'}
{'_id': ObjectId('67a01b87f2c3875d00315843'), 'user_id': 'uid5', 'username':
'John_d', 'email': 'john@go.com', 'password': 'password123', 'created_at':
'2024-01-16 10:00'}

```

Analysis of Inserting Duplicate User Data If a user with `user_id = uid5` already exists, MongoDB will allow duplicate inserts unless `user_id` is set as a unique index. If a uniqueness constraint is enforced, the insert will fail with an error.

To prevent duplicates, we should enforce a unique constraint:

```
users_collection.create_index("user_id", unique=True)
```

This ensures that inserting the same user more than once will raise an error.

Conclusion The implemented MongoDB schema successfully allows for efficient data retrieval, updates, and deletions. Key insights include:

- **Data Retrieval:** Users, posts, and comments are stored in separate collections, making queries straightforward.
- **Update & Deletion:** MongoDB operations allow efficient updates and deletions.
- **Data Integrity:** Enforcing unique constraints on `user_id` prevents duplicate entries.

MongoDB provides **scalability and flexibility**, making it a suitable choice for this blogging platform.

1.7 Step 4: Comparing MongoDB and PostgreSQL: Potential Improvements When Switching

While MongoDB was chosen for its flexibility, scalability, and document-based data structure, switching to PostgreSQL would bring certain improvements in areas where relational databases excel. Likewise, if PostgreSQL had been used, switching to MongoDB would offer different advantages. Below are the potential improvements:

1.7.1 Improvements by Switching from MongoDB to PostgreSQL

1. Stronger Data Integrity and Constraints

- PostgreSQL enforces strict **foreign keys**, **unique constraints**, and **referential integrity**, preventing duplicate or orphaned records.
- In MongoDB, relationships are managed manually, increasing the risk of inconsistent data.

2. Better Support for Complex Queries

- SQL **JOIN** operations allow efficient multi-table queries.
- MongoDB requires multiple queries or denormalization to achieve the same result.

3. ACID Transactions

- PostgreSQL ensures **atomicity**, **consistency**, **isolation**, and **durability (ACID)** for every transaction.
- MongoDB's ACID support is limited to specific scenarios and requires careful design for consistency.

4. Optimized for Structured Data

- If the dataset remains **highly structured and relational**, PostgreSQL provides better performance and consistency.
- MongoDB is more flexible but lacks built-in relational integrity enforcement.

1.7.2 Improvements by Switching from PostgreSQL to MongoDB

1. Faster Reads/Writes for Large Data Volumes

- MongoDB's **horizontal scaling (sharding)** handles massive datasets more efficiently than PostgreSQL's vertical scaling.
- NoSQL's denormalization allows faster reads without needing expensive JOIN operations.

2. Flexible Schema and Easier Updates

- MongoDB allows documents with varying structures, eliminating the need for strict predefined schemas.
- PostgreSQL requires schema modifications (**ALTER TABLE**) when adding new fields.

3. Efficient Nested Data Storage

- MongoDB's **document model** allows embedding comments directly inside posts, reducing the need for relational joins.
- In PostgreSQL, this would require additional **JOIN operations** across multiple tables, slowing down queries.

4. Better Performance in Distributed Systems

- MongoDB is optimized for **cloud-based applications** and **distributed environments**.
- PostgreSQL performs best in **centralized relational database environments**.

In conclusion, switching from **MongoDB** to **PostgreSQL** would improve **data integrity, query complexity, and transaction support**. Conversely, switching from **PostgreSQL** to **MongoDB** would enhance **scalability, flexibility, and performance in high-traffic applications**.

Ultimately, the choice depends on the project's needs: - **If strong relationships and data integrity are priorities → PostgreSQL is better.** - **If scalability, schema flexibility, and speed are priorities → MongoDB is better.**

2 Conclusion

This report successfully demonstrates the **design, implementation, and querying of a database system** for a blogging platform using MongoDB. The **data modelling process** covered conceptual, logical, and physical designs, ensuring a structured approach to handling user, post, and comment relationships. The **CRUD operations** performed validated the efficiency of MongoDB in managing semi-structured data, with queries optimized for sorting and indexing.

Furthermore, the **comparison between MongoDB and PostgreSQL** provided a critical evaluation of the advantages and trade-offs between NoSQL and SQL systems. While MongoDB offers **schema flexibility, horizontal scalability, and fast reads/writes**, PostgreSQL excels in **data integrity, complex querying, and ACID compliance**. The choice of database depends on the nature of the application and its requirements.

Overall, this implementation meets all objectives, ensuring an **efficient, scalable, and well-structured database model** for the blogging platform. The approach taken ensures adaptability for future improvements, making it a strong foundation for data management in a growing application environment.